Implementing Self-Organizing Maps (SOM) for Image Compression-

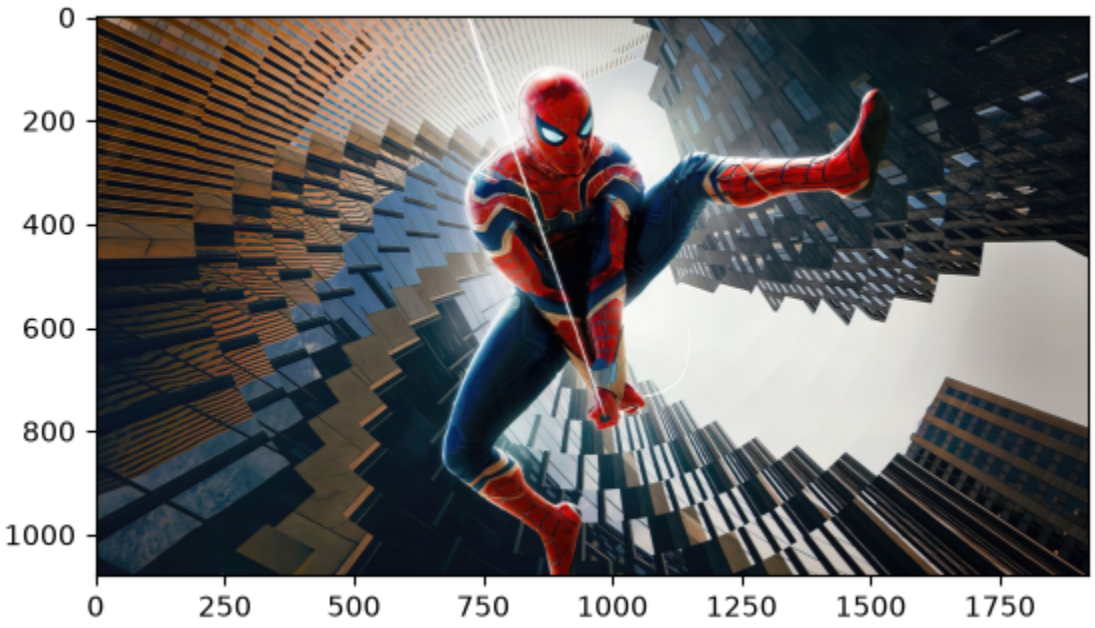
a. Implement Kohonen's Self-Organizing Map (SOM) for image compression.

b. Train SOM to reduce the number of colors in an image.

c. Display the original and compressed images to analyze quality loss

In [7]:
```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from minisom import MiniSom
from sklearn.preprocessing import MinMaxScaler
```

In [15…
```
image=Image.open("bb.jpg").convert("RGB")
pixels=np.array(image)
plt.imshow(image)
h,w,c=pixels.shape
pixels=pixels.reshape(-1,3)
scaler=MinMaxScaler()
pixels_scaled = scaler.fit_transform(pixels)
```




In [13…
```
som=MiniSom(4,4,3,sigma=1.0,learning_rate=0.5,random_seed=42)
som.random_weights_init(pixels_scaled)
som.train_random(pixels_scaled,1000)
weights=som.get_weights().reshape(-1,3)
```

In [14…
```
compressed_pixels=[]
for pixel in pixels_scaled:
    distances=np.sqrt(np.sum((weights-pixel)**2,axis=1))
```

```
    nearest=np.argmin(distances)
    compressed_pixels.append(weights[nearest])

compressed_pixels=np.array(compressed_pixels)
compressed_pixels=scaler.inverse_transform(compressed_pixels)
compressed_pixels=np.clip(compressed_pixels,0,255)
compressed_pixels=compressed_pixels.astype(np.uint8)
compressed_image=compressed_pixels.reshape(h,w,3)
```

In [16…

```
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(image)
plt.title("OG Spiderman")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(compressed_image)
plt.title("Compressed Spiderman")
plt.axis("off")

plt.show()
```

